BPKIHS Alumni Association
User Manual

Website use, administration, local running, payments, and database checks

A practical guide for alumni, staff, and local operators. Use the website for normal content work; use the database only for approved checks, maintenance, and recovery.

Prepared for the BPKIHS Alumni Association | 31 October 2026

Using this manual

Start with the section that matches your role.

The public website helps BPKIHS graduates reconnect, find alumni, attend events, share news, and support current goals. Staff use the administration area for reviewed content. Technical operators run the local service and inspect the database with read-only queries.

Contents

Part	Section	Use it for
1	Website overview	Public navigation and member journeys
2	Member account and community	Registration, sign in, profiles, posts, and events
3	Giving and payments	Donation flow, receipts, sandbox, and gateway troubleshooting
4	Administration	Content, moderation, donations, and support work
5	Run the website locally	Database, migrations, environment, development server, and checks
6	Database checks	Studio, psql, read-only inspection, backups, and safe recovery boundaries

Roles and safe boundaries

Role	Normal tools	Do not do
Visitor or alumnus	Website pages, account, directory, events, community, donate	Share another person's contact details or credentials
Content staff	Admin content, posts, approvals, event and goal editing	Edit tables directly when an admin screen supports the task
Technical operator	Terminal, migrations, health checks, database read-only queries, backups	Mark donations paid by hand or put secrets in source control

Important. Keep payment keys, database passwords, backup encryption keys, and exported personal data out of chat, screenshots, and source control.

1 Website overview

The public site is designed to make the most common alumni tasks easy to find.

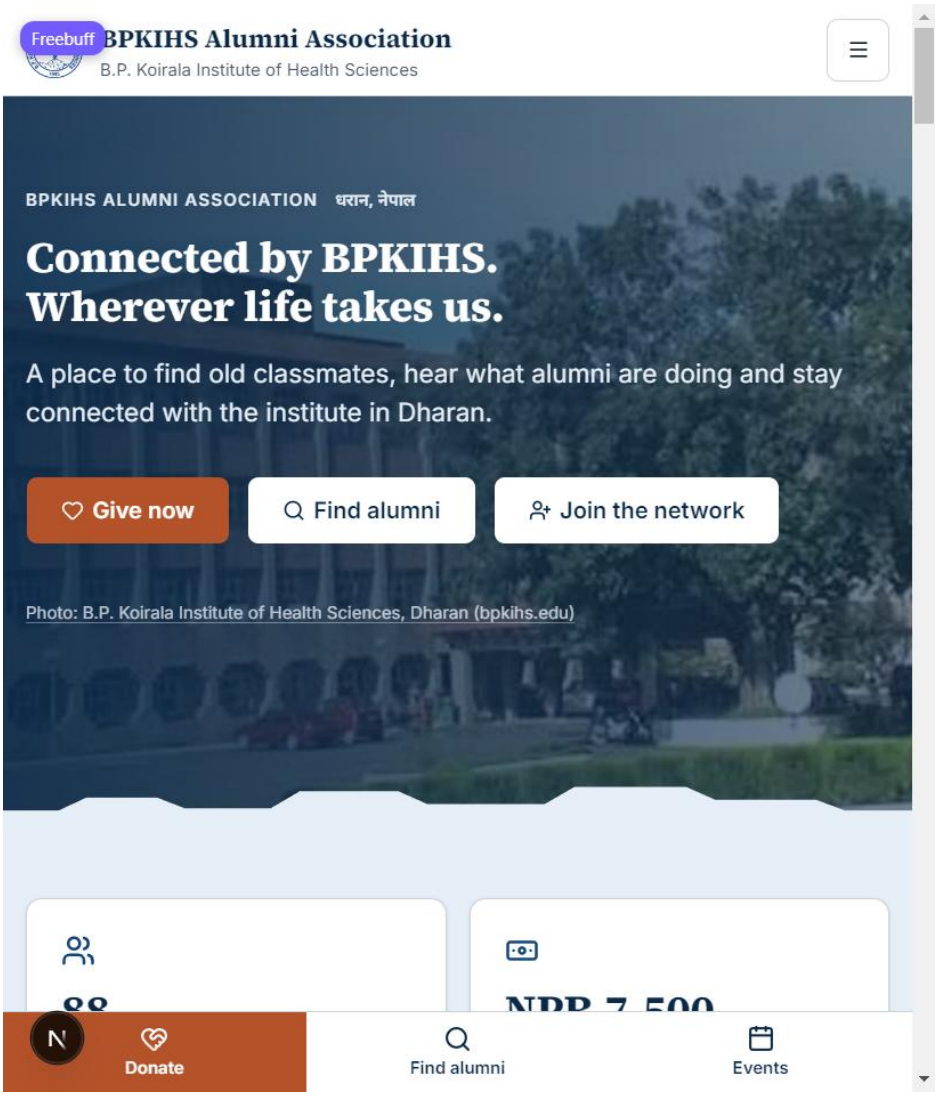


Figure 1. Home page on a mobile-sized display. The main actions are Donate, Find alumni, and Join the network.

Start from the home page

1. Open the site at the published address, or http://localhost:3000 while running locally.
2. Use the main navigation for About, Directory, Events, Community, and Support.
3. Use Donate for a general or goal-specific gift. Use Register to join as an alumnus.
4. On a phone, open the menu button to reach the same main sections.

Public page map

Page	What a visitor can do
About	Read the Association purpose, work, board information, and relationship with BPKIHS.
Directory	Search and filter approved alumni records. Visibility follows each member's privacy settings.
Events	See upcoming and past events, open event details, and RSVP when an event accepts responses.
Community	Read approved alumni stories, notices, achievements, and Association updates.
Support	Read FAQs, contact the Association, and use the helpdesk where available.
Donate	Review goals, progress, transparency information, and start a donation.

Finding what you need

- Use the site header to move between public areas without returning to the home page.
- Use the footer for Support, FAQ, legal pages, newsletter options, and Institute links.
- Open a goal, event, post, or directory profile from its card to see the detail page and related action.
- Use the browser address bar to share a public page. Do not share a signed-in account, admin, or payment-callback URL.

2 About and community pages

Use About for institutional information and Community for alumni news.

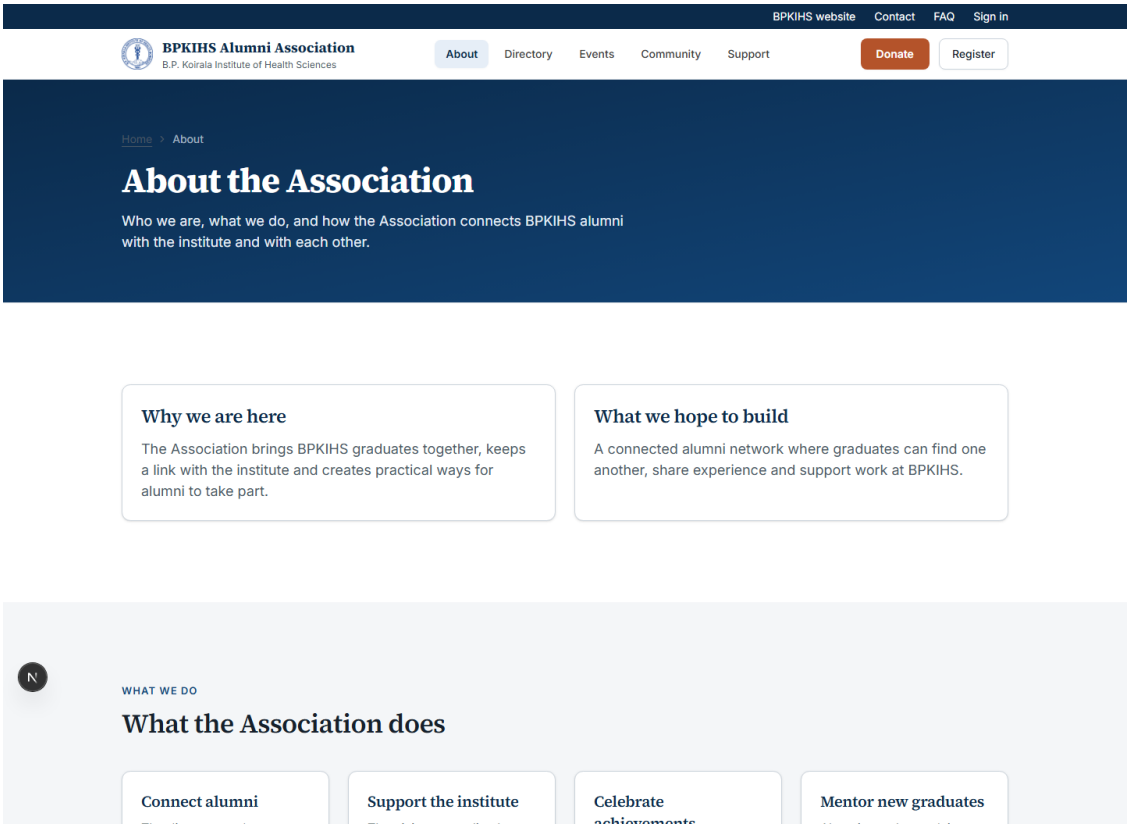


Figure 2. About page with the Association purpose, intended network, work areas, and institute connection.

About

- Open About to explain why the Association exists and how it works with BPKIHS.
- Use Meet the board to see the current leadership and available biographies.
- Use How we work with BPKIHS for governance and funds-handling information.

Community, directory, and events

Read approved updates, find classmates, and RSVP for gatherings.

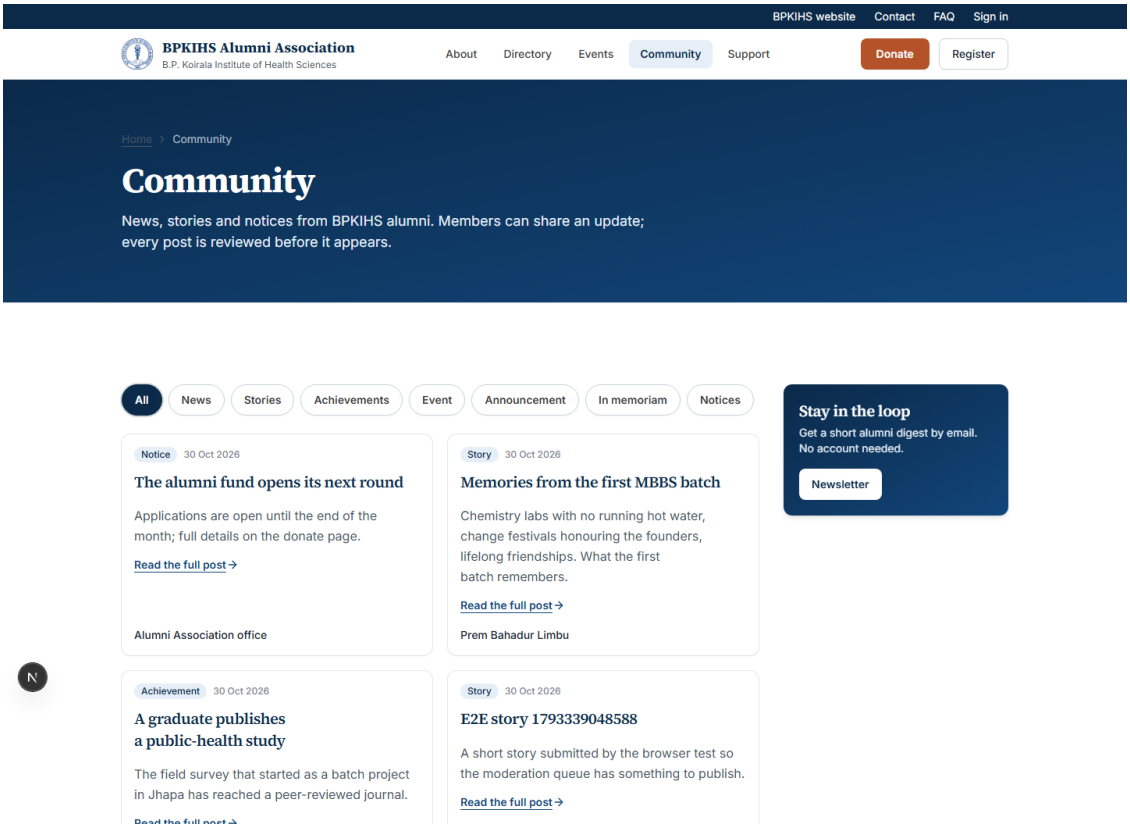


Figure 3. Community page. Visitors can read approved updates; signed-in alumni can use the sharing flow when it is enabled.

- Community posts are moderated. Submitted content is not public until approved.
- The directory exposes only fields the alumnus has allowed for the viewer's role.
- For an event, open the detail page to check date, venue, mode, capacity, and RSVP availability.

3 Member account and profile

Members use one account for their profile, privacy choices, and alumni participation.

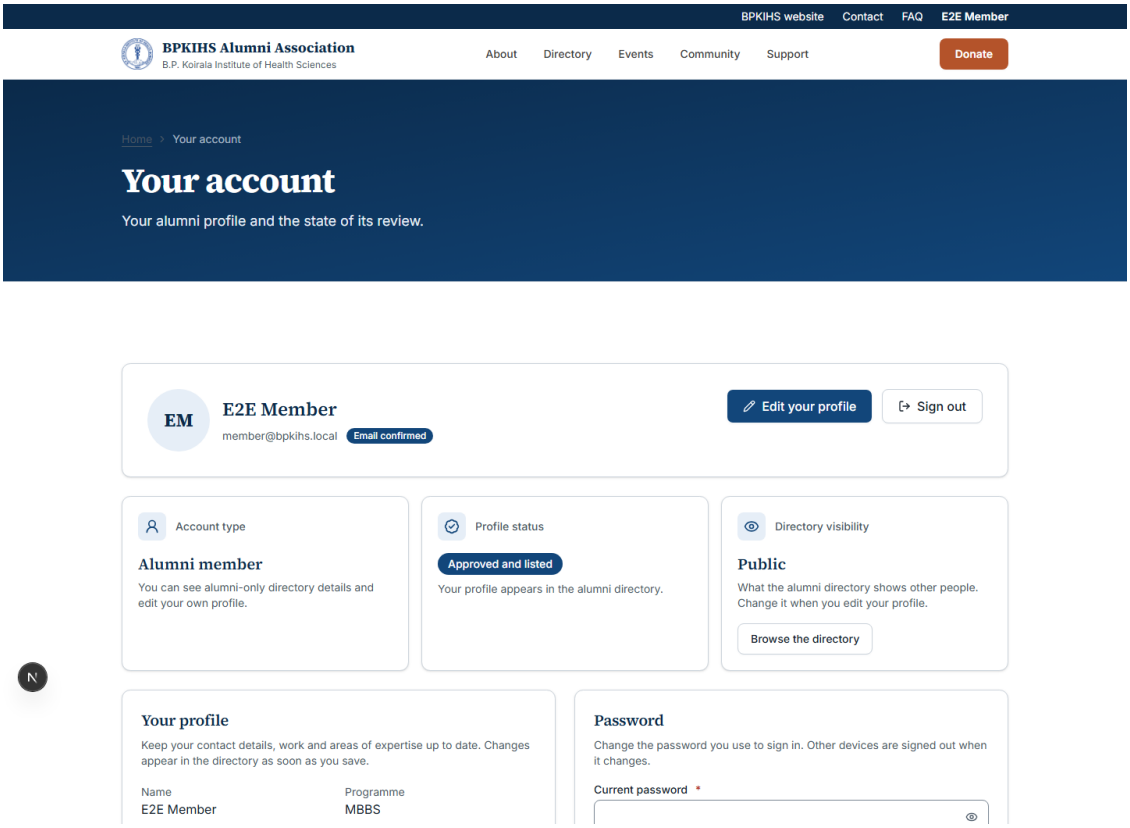


Figure 4. Signed-in member account page. Fields and options vary by the member's current profile and approval status.

Register and sign in

5. Select Register from the top navigation and complete the membership registration flow.
6. Use the verification message or code when prompted. A submitted alumni profile may need staff approval before it appears in the directory.
7. Use Sign in to return to the account. Use the forgotten-password flow if a password is unavailable.
8. Open Account to review identity, update profile information, and manage settings that are available to the signed-in role.

Privacy and profile checks

- Choose profile, email, and phone visibility carefully. A public directory record should never expose information that the member has not selected for viewing.
- Before publishing a new profile photo, confirm the alumnus has consented and that the image is appropriate for an alumni directory.
- Staff should review a registration before approval, especially imported or incomplete records.

4 Giving and payments

The site records a gift only after a payment gateway confirms it.

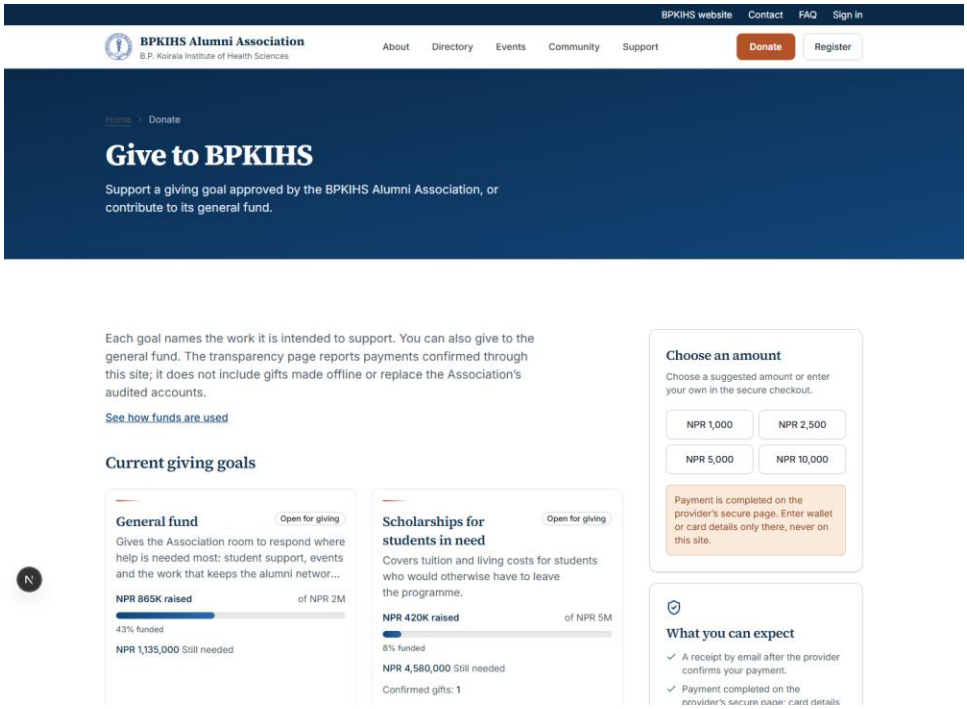


Figure 5. Donate page with goal cards, progress, and supporting information. Content reflects the current database.

Make a donation

9. Open Donate and choose a goal, a quick amount, or the general-fund option.
10. Sign in if prompted. The site creates a pending donation before redirecting to the selected payment gateway.
11. Review the gateway amount and merchant information, then complete or cancel the transaction at the gateway.
12. After a verified callback, the site records a succeeded donation, updates the goal total, and sends or logs a receipt according to the email configuration.

Payment controls

- A browser return alone is not proof of payment. The server verifies the gateway response before marking a donation as succeeded.
- Do not set donations.status to succeeded directly in SQL. It bypasses the payment event, goal-total, and receipt flow.
- For sandbox testing, use test credentials only in the gateway's test environment. Do not place merchant secrets in this manual or in source code.
- If a donor cancels or closes the gateway, the pending donation is not a completed gift.

5 Administration

Staff members with the admin role can manage approved content and operational queues.

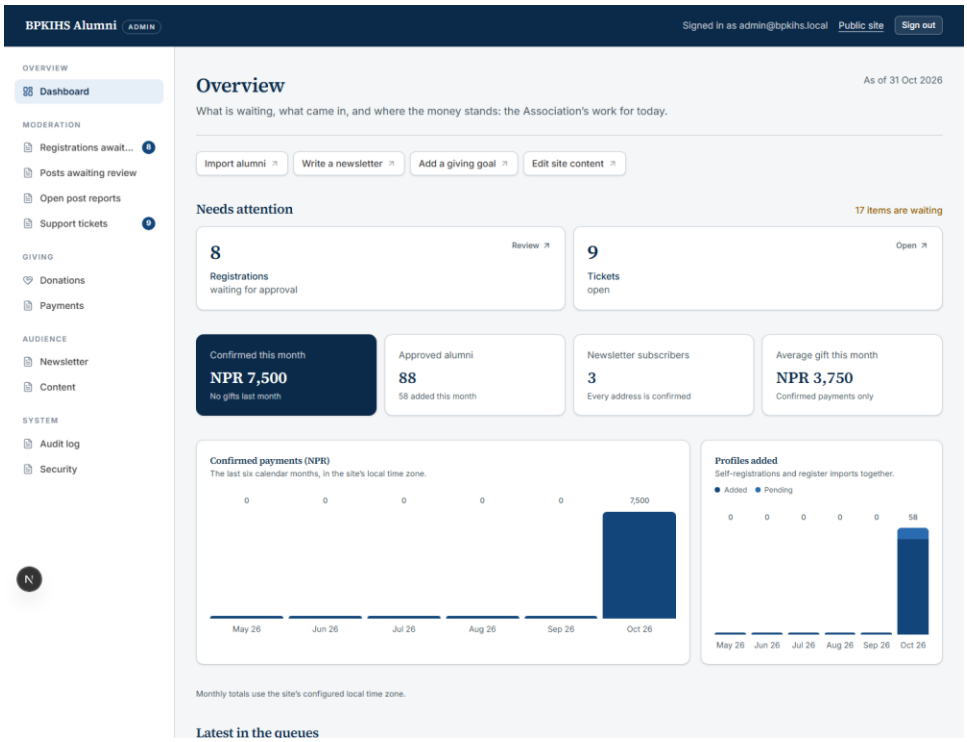


Figure 6. Admin dashboard in a local fixture environment. Production counts and menu items reflect the live data and staff permissions.

Routine staff work

Area	Typical task	Check before saving
Content	Add or update events, giving goals, goal updates, board records, FAQs, and welcome text.	Dates, public wording, links, image rights, and publish state.
Posts	Review a pending community post, then approve or reject it.	Accuracy, respectful language, consent, and image suitability.
Alumni	Review registrations, imports, and requests that affect directory visibility.	Identity, programme/batch data, profile privacy, and photo consent.
Donations	Review the ledger, goal totals, and gateway callback records.	Gateway status and payment event; never manufacture a paid result.
Support	Review tickets and follow up using the reference number.	Avoid putting personal data into public replies.

Admin review checklist

- Use the admin screen rather than a direct database edit whenever that screen supports the action.
- Confirm public text, dates, and external links before publish.
- Do not paste payment secrets, password-reset tokens, or full donor exports into content fields or support replies.
- Record the reason for a rejection when it will help the submitter correct the issue.

6 Run the website locally

A development environment uses Next.js, PostgreSQL, and the environment variables in .env.local.

First-time setup

13. Install the project dependencies from the repository root.
14. Copy .env.example to .env.local and set required values. Keep .env.local out of version control.
15. Start Docker Desktop, then start PostgreSQL and apply the migrations.
16. Create an administrator if this is a new development database.
17. Start the development server and browse to http://localhost:3000.

```
npm install
Copy-Item .env.example .env.local
npm run db:up
npm run db:migrate
npm run db:admin
npm run dev
```

Daily run and quality checks

Command	Use it for
npm run dev	Start Next.js locally after environment and database are ready.
npm run check:env	Report missing or unsafe environment configuration before a release.
npm run typecheck	Validate TypeScript and Next route types.
npm run lint	Run the repository lint rules.
npm run test	Run the unit-test suite.
npm run test:e2e	Run browser workflows against fixture data.
npm run check:health	Check the running application health endpoint.

Tip. Restart npm run dev after changing server-side environment values. The local site must use the same APP_URL and port expected by its authentication and callback configuration.

7 Check the database safely

Use the database to inspect state and reconcile operations. Prefer read-only queries first.

Open the database

- Use npm run db:studio for a local browser-based view of the schema and rows.
- Use psql in the database container for controlled SQL checks.
- The local database service is named bpkihs-postgres and is exposed on localhost port 5433 by default.

```
npm run db:studio

docker exec -it bpkihs-postgres psql -U bpkihs -d bpkihs
```

Read-only queries you will use

Run these in psql to inspect operational state. They do not modify rows.

```
-- Pending alumni registrations
select id, full_name, email, batch_year, programme, created_at
from alumni
where status = 'pending' and source = 'self_registered'
order by created_at desc
limit 20;

-- Recent payment state
select id, amount_minor, method, status, gateway, gateway_ref, created_at
from donations
order by created_at desc
limit 20;

-- A donation and its callback events
select d.id, d.status, d.gateway_ref, p.event_type, p.created_at
from donations d
left join payment_events p on p.donation_id = d.id
where d.id = '<donation id>';

-- Current goal totals
select title, target_minor, raised_minor, active
from donation_goals
order by sort_order;
```

Reading money values

Remember. Money columns ending in _minor store paisa as whole numbers. NPR 5,000.00 is stored as 500000. Convert before reporting an amount to a person.

8 Database maintenance and backups

Direct writes are a repair tool, not the normal way to operate the site.

Safe order of operations

18. Use the website's admin screen if it can perform the action.
19. If a direct database repair is necessary, take a fresh backup first and write down the exact target row and intended result.
20. Start with a SELECT query that proves the target. Do not use broad updates, wildcards, or a missing WHERE clause.
21. After a change, re-run the relevant read-only check and record what was changed and why.

Backups

```
npm run db:backup
# Optional custom backup location
BACKUP_DIR=/var/backups/bpkihs ./scripts/backup-db.sh
```

- The backup script writes a compressed SQL dump and keeps the configured retention period.
- Store production backups away from the application server and encrypt them at rest.
- Practice a restore into a separate scratch database before relying on a backup process.
- A reset command such as docker compose down -v destroys the local database volume. Treat it as destructive.

Never do these things

- Never manually mark a donation succeeded. Only the verified gateway callback completes a gift.
- Never edit a migration that has already been applied. Create a new forward-only migration.
- Never copy donor emails, payment payloads, or entire exports into public documents or issue trackers.
- Never put database passwords, encryption keys, or gateway keys in the repository.

9 Payment operations and troubleshooting

Gateway configuration determines whether a payment method is offered at checkout.

Sandbox test flow

22. Set PAYMENT_MODE=sandbox and provide only sandbox gateway configuration in .env.local.
23. Restart the development server, sign in as a test user, choose a goal and an amount, then select a configured gateway.
24. Complete the test entirely at the gateway. The website must receive and verify the callback before it records success.
25. Check the newest donation, the payment_events table, the goal total, and the receipt log or email result.

When eSewa shows an unavailable service message

The eSewa sandbox can return an upstream availability or timeout message even when the request reaches its form correctly. Confirm that the application is using sandbox mode and the test merchant configuration, then retry later or with another permitted test account. Do not switch to production keys or URLs to solve a sandbox outage.

Quick diagnosis

Symptom	First check
Payments are not configured	PAYMENT_MODE is sandbox or live, and at least one gateway has its required variables.
eSewa is not configured	ESEWA_PRODUCT_CODE and ESEWA_SECRET_KEY are set only in the server environment.
Callback never arrives	APP_URL is a reachable public HTTPS origin in production and the correct callback URL is registered with the gateway.
Donation stays pending	The user abandoned the gateway or a verified callback has not arrived. Inspect callback events before acting.
No receipt	Check email transport configuration and server logs; a receipt failure must not change a verified donation result.

10 Release and support checklist

Use this checklist before publishing a code or content change.

Before deployment

- Run npm run check:env and resolve all production blockers.
- Run npm run typecheck, npm run lint, and npm run test. Use end-to-end tests for high-risk flows such as authentication, donations, and moderation.
- Take a database backup before a migration or material configuration change.
- Apply production migrations with npm run db:migrate:prod, not by editing live tables manually.
- Confirm APP_URL, contact details, email configuration, and gateway return URLs match the real public domain.

After deployment

- Run npm run check:health and open the home page, directory, events, community, and donate pages in a browser.
- Submit one harmless support or newsletter test only if the environment is designed for it; avoid sending test mail to the real audience.
- For payment releases, perform a small authorised end-to-end gateway payment and reconcile it with the gateway dashboard and the donation ledger.
- Watch server logs for callback, email, and authentication errors after the release.

Where to find more detail

Use docs/DB-AND-PAYMENTS.md for gateway and database procedures; docs/HOWTO.md for daily administration; docs/DEPLOYMENT.md for releases; docs/SECURITY.md for controls; and docs/BACKEND.md for architecture.

Quick reference

```
Start database:      npm run db:up
Apply migrations:    npm run db:migrate
Start website:       npm run dev
Check environment:   npm run check:env
Run tests:           npm run test
Browse database:     npm run db:studio
Create backup:       npm run db:backup
Check health:        npm run check:health
```